RTL Debugging Agent for Out-of-Order Processor Bugs

Vayun Gupta

Question

This project studies a practical hardware-debug question: when an LLM is asked to repair broken RTL from a real processor codebase, which debug artifacts make the repair more reliable? The controlled comparison is between three evidence conditions: source-level RTL context only, RTL plus simulator failure logs, and RTL plus simulator logs plus a short waveform-style signal summary.

This matters because real RTL debugging is rarely a clean code-generation task. An engineer usually starts from a failing simulation, a failing assertion, a waveform, and a large set of plausible files. The useful question is therefore not whether an LLM can write a module from scratch, but whether realistic verification evidence helps it localize and repair the kind of temporal bugs that appear in processor development.

Prior Prediction

Before running the substantial experiments, I wrote the dated prediction file `predictions.md` on 2026-05-14. The primary prediction was that simulator logs would improve repair success over source-only repair, and that adding waveform-derived signal summaries would help most on temporal or protocol-level bugs. In particular, I expected the trace summaries to matter most when the root cause was a relation across cycles, such as a same-cycle rename bypass, a multi-beat memory transaction, or a branch-recovery state transition.

System Overview

The project implements a small verification-guided RTL repair benchmark and agent loop. The pipeline is:

Stage	Engineering role
Bug mining	Inspect ECE 411 processor commit history and identify real bug-fix commits with local, testable RTL behavior.
Bug injection	Start from the current known-good Verilator-compatible RTL and apply a targeted mutation that recreates the historical bug.
Validation	Run the reference and buggy versions through the HW4-style verifier; keep a case only if reference passes and buggy fails.
Artifact generation	Produce simulator failure logs and a concise trace summary that names the relevant signal relationship.
Repair	Ask the model to patch the RTL under one evidence condition. In localization mode, it sees multiple plausible files.
Verification	Compile and run the repaired module against the hidden Verilator testbench and record pass/fail JSON logs.

The implementation reuses the HW4 verifier interface and per-trial JSON format, but the problem suite is new. The final-project code is organized as follows: `bug_cases.py` defines the injected bugs and testbenches; `artifacts.py` builds the debug artifacts; `repair_agent.py` constructs the prompts and checks model patches; and `evaluate.py` runs the controlled study and writes auditable per-trial logs.

Benchmark Construction

The most industry-relevant part of the project is the benchmark construction. Rather than relying only on synthetic mistakes, I mined a private ECE 411 out-of-order processor repository for bug-fix commits, then converted selected fixes into controlled repair cases. A case was included only when the bug could be isolated to a small RTL behavior and tested under Verilator without relying on the original Synopsys environment.

Case	Commit	Recreated failure
C1	`bbb61a8`	Instruction burst adapter starts a memory transaction even when the instruction cache lost arbitration.
C2	`513f159`	RAT read path misses a same-cycle speculative destination bypass.
C3	`582bf6c`	Dispatch observes a stale registered ROB index instead of the current tail index.
C4	`2c76489`	Branch recovery restores a speculative freelist checkpoint instead of the committed shadow state.
C5	`5ae5544`	JALR target update masks the immediate but omits the resolved `rs1` base.

These real bugs are supplemented by seven synthetic stress cases that cover similar RTL-debug patterns: FIFO flush priority, payload integrity, multi-beat burst timing and packing, RAT commit/flush interaction, freelist resource accounting, and ROB completion ordering. The main result in this report focuses on the five real-history cases because they are the best match to the final project goal of moving beyond a clean benchmark toward realistic engineering use.

Controlled Comparison

The main experiment uses the real-history cases in fault-localization mode. For each bug, each condition is run for three trials, giving 45 total repair attempts. The only intended difference between conditions is the evidence given to the model:

Condition	Evidence available to the repair agent
`rtl_only`	Candidate module interfaces and the need to repair RTL, but no failure reason, design intent, or trace clue.
`sim_log`	Problem statement, success criteria, candidate RTL files, and the simulator failure log.
`sim_log_trace`	Everything in `sim_log`, plus a short waveform-style summary of the relevant signal timeline.

This is not a claim that the three conditions are equally informative to a human engineer. They are intentionally different information budgets. The goal is to measure whether adding realistic verification artifacts changes the model's ability to repair temporal RTL bugs.

Validation

The verifier is the main guardrail against confusing execution with success. Every case is checked in two directions before being used: the reference RTL must pass the hidden testbench, and the injected buggy RTL must fail with the expected symptom. The current real-case validation artifact, `real_case_validation.json`, shows 5/5 references passing and 5/5 buggy versions failing.

The repair metric is strict: a trial counts as a pass only if the model-produced SystemVerilog compiles and passes the hidden Verilator testbench. Syntax errors, interface mismatches, partial patches, and behaviorally wrong repairs all count as failures. The per-trial results are stored in `repair_results.json`, including the case id, condition, trial number, failure reason, model confidence, and prompt size.

Results

The trace-summary condition was the strongest. Across the five real-history bugs and three trials per condition, the source-only baseline repaired 6 of 15 trials, simulator logs repaired 8 of 15, and simulator logs plus trace summaries repaired all 15.

Condition	Passes / Total	Pass rate
RTL only	6 / 15	40.0%
Simulation log	8 / 15	53.3%
Simulation log + trace summary	15 / 15	100.0%

Case	RTL only	Sim log	Log + trace
C1: i-cache arbitration	3/3	3/3	3/3
C2: RAT rename bypass	0/3	0/3	3/3
C3: ROB dispatch index	3/3	3/3	3/3
C4: freelist recovery	0/3	0/3	3/3
C5: JALR target	0/3	2/3	3/3

Discussion

The result supports the prior prediction, but with an important nuance. Two bugs were easy in every condition: the instruction-cache arbitration bug and the stale ROB dispatch index. These failures are local enough that the model can often infer the patch from the interface and nearby RTL structure alone.

The more interesting cases were the temporal state bugs. The RAT bypass and freelist recovery bugs both failed all source-only and log-only trials, but passed all trace-summary trials. In these cases, the simulator failure reason named the symptom, but not the state relationship that had to be preserved. The trace summary made the missing invariant explicit: same-cycle speculative rename mapping must bypass the RAT table, and branch recovery must restore the freelist from committed shadow state rather than a stale speculative checkpoint.

The JALR target bug sits between those extremes. Simulator logs were enough in two of three trials because the failure message pointed at the target address, but trace summaries made the repair consistent across all trials by spelling out the relation between the stored immediate, the resolved `rs1` base, and the final masked target.

From an engineering perspective, this suggests that the valuable artifact is not merely "more text." The useful artifact is a compact statement of the cycle-level invariant that the waveform demonstrates. That is close to how a hardware engineer uses a waveform: not as a dump of every signal, but as evidence for a small causal story about state, timing, and protocol.

Limitations and Threats to Validity

The strongest argument against the conclusion is sample size. The main real-history study has five bugs, which fits the guideline's recommended 5–20 carefully chosen cases, but it is still a curated subset of one processor project. The result should therefore be read as a controlled characterization, not a universal claim that trace summaries always make RTL repair succeed.

The trace summaries are hand-authored from targeted testbench behavior rather than automatically extracted from VCD files. This is useful for a clean course experiment because the evidence is auditable, but it understates the difficulty of building a production waveform summarizer. A deployable version would need to extract candidate signal timelines automatically and avoid leaking the repair in overly direct language.

Another limitation is that the repair model sees a small set of candidate files in localization mode, not an entire industrial RTL tree. The benchmark is more realistic than exact-module reconstruction, but still much smaller than a commercial verification environment. Finally, the pass/fail verifier checks the targeted hidden behavior; it is possible for a patch to pass this test while remaining incomplete for the full processor.

Reproducibility

The project includes the source code, dated prediction file, generated result JSON, and validation artifacts required to audit the comparison. No API keys are stored in the submitted files; the OpenAI key is read from the environment.

To validate the bug suite:

```
cd FinalProject
python3 evaluate.py --validate-cases-only --real-only \
  --output-json real_case_validation.json
```

To reproduce the main real-history fault-localization study:

```
python3 evaluate.py --real-only --localize --resume --trials 3 \
  --conditions rtl_only sim_log sim_log_trace \
  --output-json repair_results.json
```

The expected output is a JSON file with 45 real-history trials and aggregate condition results matching the table above. The main known failure mode is API token-rate pressure on long prompts; `evaluate.py` supports `--resume` and prompt shrinking so interrupted runs can continue without rerunning completed trials.